

第14讲：Loss Function

模型到底怎样知道自己“错了多少”？

- 把 Prediction、Target、Error 与 Loss 彻底区分开。
- 理解为什么 Loss 往往要把很多错误压缩成一个标量。
- 区分单样本 Loss、Batch Loss、Sum 与 Mean。
- 从 $\text{Loss}(w)$ 建立下一阶段 Gradient / Optimizer 的桥梁。
- 连接 LLM 的 Next-token Prediction Loss，但暂不提前展开 Cross Entropy。

本讲最低目标

学完以后，你必须能区分 Prediction、Target、Error、Loss 和 Batch Loss，并解释：为什么训练需要一个“可优化的标量目标”，为什么带符号 Error 不能直接拿来训练。

1. 为什么 Forward 之后还不够？

【原理】

到第12讲，我们已经能完成 Forward；第13讲又知道“导数衡量局部变化率”。但训练仍缺一个核心对象：模型现在到底有多错？

如果没有一个数字衡量预测质量，就不知道参数应该朝哪个方向优化。这个数字通常由 Loss Function 产生。

最简训练链条可以写成：

Input -> Model -> Prediction -> Compare with Target -> Loss

只有得到 Loss 以后，后面才有意义去问：

if Weight changes a little, how does Loss change?

【通俗解释】

把模型看成学生答题。Prediction 是学生答案，Target 是标准答案；Error 是“答偏了多少”，Loss 则是老师按照某套评分规则，把错误转成一个可以比较和优化的分数。

2. 五个最容易混淆的词

概念	通俗含义	例子
Prediction / $\hat{y}$	模型给出的预测	3.0
Target / y	希望模型得到的正确答案	4.0
Error	预测与目标的差，可带正负号	$3-4=-1$
Loss	按照某种规则把错误变成训练代价	$(-1)^2=1$
Objective	真正被优化的总体目标，常由许多 loss 聚合并可能加正则项	$\text{mean}(\text{loss})+\text{regularization}$

术语提醒

论文里 loss / objective / training objective 有时会被宽松混用。科研阅读时不要死抠名字，要看作者究竟在最小化/最大化什么公式、在哪个维度做了平均、是否还加了其他项。

3. 为什么不能直接平均 Error ?

具体数字例子

两个样本的 Target 都是 4 :

```
prediction_1 = 2 -> error_1 = -2
```

```
prediction_2 = 6 -> error_2 = +2
```

如果直接平均带符号 Error :

```
mean(error) = (-2 + 2) / 2 = 0
```

得到 0 看起来像 “没有错误”，但两个预测其实都错了 2。

【核心结论】

Error 只是 “偏向哪边、偏了多少” 的原始差值；Loss 需要采用合适规则，避免正负错误互相抵消，并把 “错得多严重” 编码成优化信号。

最简单的处理方式包括取绝对值或平方。正式的 MSE 会在下一讲系统学习。

4. 一个 Loss Function 到底做了什么？

可以把单样本 Loss 写成：

```
l = loss(y_hat, y)
```

其中：

```
y_hat = prediction
```

```
y = target
```

```
l = this sample's loss
```

例子：平方误差（先用来建立直觉）

```
l = (y_hat - y)^2
```

如果 $y=4$ ：prediction=3 时 loss=1；prediction=4 时 loss=0；prediction=7 时 loss=9。

为什么平方很方便？

① 正负 Error 不会抵消；② 预测离目标越远，Loss 越大；③ 大误差被更强地惩罚；④ 公式连续、方便后续做梯度计算。下一讲会正式讲 MSE。

5. Loss 并不是“世界真相”，而是一套训练规则

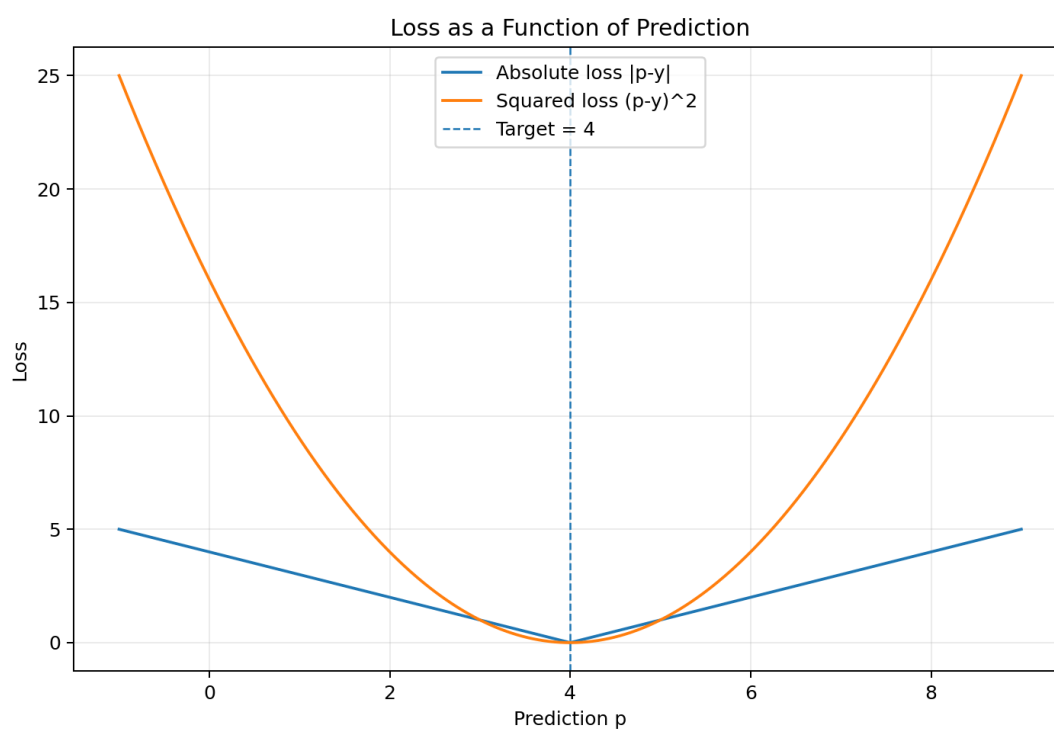
同一个 Error，可以使用不同 Loss。以绝对误差 $|e|$ 和平方误差 e^2 为例：

Error	Absolute Loss	Squared Loss
0.5	0.5	0.25
1	1	1
2	2	4
4	4	16

误差从 2 增大到 4：Absolute Loss 只变成 2 倍，而 Squared Loss 从 4 变成 16，变成 4 倍。

科研洞察

选择什么 Loss，会改变“哪些错误被认为更严重”，也就改变优化过程。论文提出一个新 Loss 时，真正应该问的是：它改变了什么训练偏好？为什么这种偏好与任务目标一致？



6. 单样本 Loss 与 Batch Loss

真实训练通常不会一次只看一个样本，而是一个 Mini-batch。假设 Batch 中有 B 个样本，每个样本得到一个 loss：

$$l_1, l_2, \dots, l_B$$

常见做法是取平均：

$$L_{\text{batch}} = (1/B) * \sum_i l_i$$

具体数字例子

若四个样本的 squared loss 是：

$$[1.00, 1.00, 0.25, 1.00]$$
$$\text{sum} = 3.25$$
$$\text{mean} = 3.25 / 4 = 0.8125$$

为什么 Mean 很常见？

如果把同样的数据复制一遍，样本数翻倍，Sum 会翻倍，但 Mean 仍保持 0.8125。这样 Loss 的数值尺度不容易仅因为 Batch Size 改变而整体翻倍。

7. Sum 与 Mean：没有“永远正确”，必须看定义

框架中常见 reduction 选项：

reduction = "none" -> keep every element/sample loss

reduction = "sum" -> add them together

reduction = "mean" -> average them

科研复现实验时，必须确认论文或代码到底采用哪一种聚合方式。因为这会影响 Loss 数值尺度，进一步影响梯度尺度与 Learning Rate 的有效大小。

常见坑

不要只看论文写“Cross Entropy Loss”。还要看：是按 sample 平均、按 token 平均、按 sequence 平均，还是 sum？是否 mask 掉 padding？这些细节都可能影响训练行为。

8. Loss 为什么通常最后要变成一个标量？

一个 Batch 可能有成千上万个预测数字，但优化器最终需要一个明确的“总体方向”。常见训练流程会把大量局部错误聚合成一个标量 Loss：

many predictions -> many local losses -> one scalar objective

然后问：

$dL/dw_1, dL/dw_2, \dots, dL/dw_n$

也就是“这个总体 Loss 对每个参数有多敏感”。

关键桥梁

第13讲学的是“输出对参数的导数”；从现在开始，真正训练关心的是 Loss 对参数的导数。Loss 把“模型表现”变成了可以求梯度的目标。

9. Loss 作为参数的函数：第一次看到微型 Loss Landscape

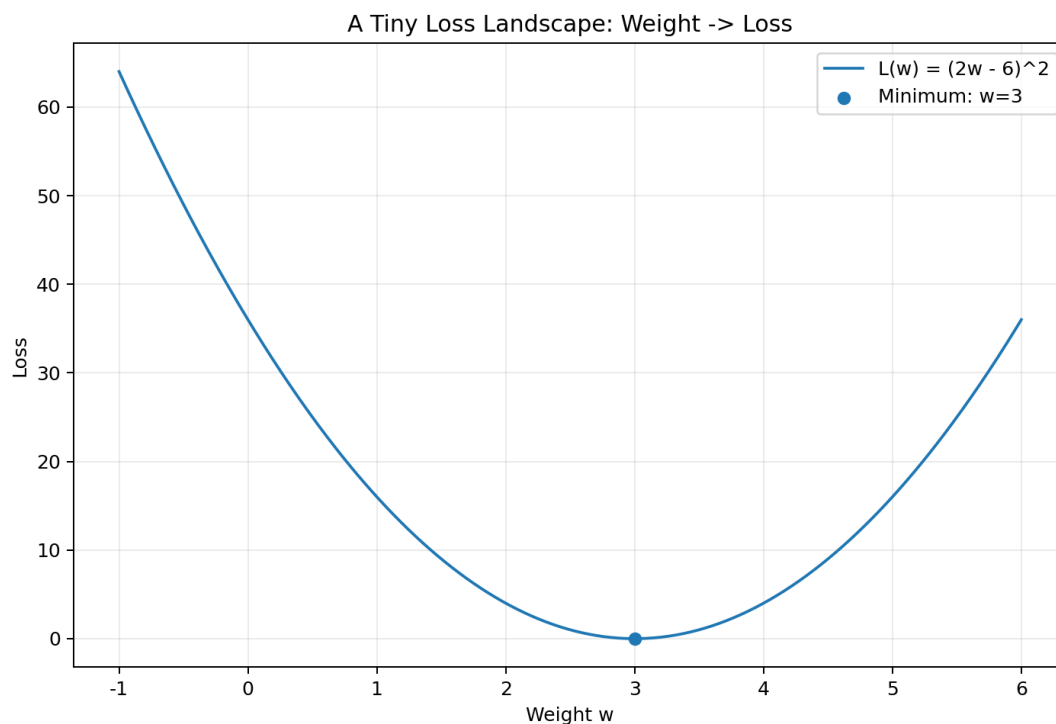
考虑一个只有一个 Weight 的模型：

$\text{prediction} = w * x$

固定 $x=2$, Target=6, 并使用 squared loss：

$L(w) = (2w - 6)^2$

不同 w 会得到不同 Loss： $w=2 \rightarrow \text{prediction}=4 \rightarrow \text{loss}=4$ ； $w=2.5 \rightarrow \text{loss}=1$ ； $w=3 \rightarrow \text{loss}=0$ 。



这张图以后会反复出现

横轴是参数 w ，纵轴是 Loss。训练就是不断调整 w ，让 Loss 尽量下降。下一阶段的 Gradient Descent / Optimizer，本质上都在回答“怎么沿这张地形往低处走”。

10. Loss 与 Metric 不是一回事

【原理】

Loss 主要服务于训练优化；Metric 主要服务于评价和解释。它们可以相同，也可以不同。

例如分类任务可能用 Cross Entropy 训练，却用 Accuracy、F1 等指标报告结果。为什么不直接拿 Accuracy 当 Loss？一个原因是 Accuracy 对小幅参数变化往往不连续、没有细腻梯度信号。

科研时必须分清：

`training objective != evaluation metric (not always the same)`

论文阅读问题

看到“performance improves”时，先确认作者说的是训练 Loss 下降，还是验证集 Accuracy/F1/Reward 等真正任务指标提升。Loss 下降不自动保证所有外部指标都更好。

11. 一个好 Loss 需要考虑什么？

当前阶段不用掌握 Loss 设计理论，但至少建立四个判断维度：

- 1) 方向正确：预测更接近目标时，Loss 应该倾向更小。
- 2) 训练信号：参数轻微改变时，Loss 最好能提供有用的局部变化信息。
- 3) 尺度与聚合：Batch、Token 数量改变时，数值尺度是否合理。
- 4) 任务一致性：Loss 惩罚的错误，是否真的是任务在意的错误。

不是所有 Loss 都必须处处光滑

深度学习中也会出现不可导点或使用次梯度/约定梯度的情况。当前不要把“Loss 必须完美可导”背成死规则；更重要的是理解为什么基于梯度的训练偏好可提供稳定局部信号的目标。

12. 与 LLM / Transformer 的直接关系

12.1 Next-token Prediction

LLM 预训练时，输入前面的 Token，模型输出“下一个 Token”的预测分布；真实下一个 Token 就是 Target。

```
context -> Transformer -> token prediction -> compare with true next
token -> token loss
```

很多 Token 的 loss 再聚合成训练 Loss。具体使用的 Cross Entropy 会在第17-18讲系统拆开。

12.2 为什么 Token Mask 很重要

如果序列里有 padding 或某些位置不应该参与训练，就需要 mask 掉对应位置。否则“平均 Loss”的分母和参与项都会改变。以后读训练代码时，这类细节非常重要。

12.3 Fine-tuning / LoRA

不管全参数训练还是 LoRA，只要使用监督式目标，通常仍然是：Forward -> Loss -> Backward，只是哪些参数允许更新不同。

12.4 Agent Systems

Agent 系统的最终评价可能是任务成功率、成本、延迟、工具调用正确率等；这些指标不一定直接等于底座模型训练 Loss。做 Agent 研究时要避免把“模型内部 Loss”与“系统级性能指标”混为一谈。

13. 代码实验：亲手观察 Loss

```
target = 4.0
prediction = 3.0

error = prediction - target
loss = error ** 2

print(error) # -1
print(loss) # 1
```

Batch 情况：

```
targets = np.array([3.0, 5.0, 2.0, 8.0])
preds = np.array([2.0, 6.0, 2.5, 7.0])

per_sample_loss = (preds - targets) ** 2
batch_loss = per_sample_loss.mean()

print(per_sample_loss)
print(batch_loss)
```

完整实验还会测试 Error 抵消、Sum vs Mean，以及 Loss(w) 曲线。

14. 常见误区

误区 1：Error 和 Loss 是一回事

Error 通常是原始差值；Loss 是把 Prediction 与 Target 映射为训练代价的函数。

误区 2：Loss 越小，现实任务一定越好

不一定。训练 Loss、验证 Loss、系统级指标可能不完全一致；还可能过拟合。

误区 3：Loss 的绝对数值可以跨任务直接比较

通常不能。不同 Loss 定义、reduction、数据尺度会产生完全不同的数值范围。

误区 4：Mean 和 Sum 只是显示方式

不是。它们改变目标尺度和梯度尺度，可能影响训练超参数。

误区 5：Loss Function 是模型自动决定的

通常是研究者/训练方案事先设计或选择的。它编码了“我们希望模型优化什么”。

15. 本讲最低验收标准

必须真正掌握

① 能区分 Prediction、Target、Error、Loss；② 能解释为什么带符号 Error 会抵消；③ 能手算单样本平方误差；④ 能区分 per-sample / sum / mean；⑤ 能解释为什么训练通常需要一个标量 Objective；⑥ 能说清 Loss 与 Metric 不一定相同；⑦ 能把 Loss 看成参数的函数。

一句话总图

Model -> Prediction -> Loss -> Gradient -> Parameter Update

下一讲预告

第15讲将正式学习 MSE：为什么平方、为什么平均、Shape 如何变化，以及 MSE 的梯度直觉。